PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Instituto de Ciências Exatas e de Informática

Teoria dos Grafos e Observabilidade: análise comparativa de arquiteturas *backend* sob estresse de viralização*

Gabriel Assunção Costa[1]
Matheus Henrique Resende Magalhães[2]
Bernardo Jeunon de Alencar[3]

Resumo

O resumo é redigido em parágrafo único. Deve apresentar, de forma sucinta, o contexto, o problema, a justificativa, o objetivo, o desenvolvimento, os resultados e a conclusão do documento. O texto deverá conter de 100 a 250 palavras.

Palavras-chave:

Abstract

The abstract must be written in a single paragraph and should briefly present the context, problem, justification, objective, development, results, and conclusion of the document. It should contain between 100 and 250 words.

Keywords:

*Artigo apresentado ao Instituto de Ciências Exatas e Informática da Pontifícia Universidade Católica de Minas Gerais, campus Betim, como pré-requisito parcial para obtenção do título de Bacharel em Sistemas de Informação.

[1]Aluno do curso de Sistemas de Informação.
[2]Aluno do curso de Sistemas de Informação.
[3]Orientador do trabalho.

1 INTRODUÇÃO

Redes sociais permitem que conteúdos se propaguem rapidamente entre usuários e produzam variações intensas no volume de interação. A disseminação decorre das características do conteúdo e das decisões sucessivas de compartilhá-lo, enquanto a posição do usuário na rede condiciona o alcance potencial de cada publicação. Quando um conteúdo alcança usuários com grande capacidade de disseminação, compartilhamentos sucessivos podem gerar picos de requisições, processamento e persistência. A estrutura dessas cascatas não se reduz ao total de compartilhamentos, pois a dinâmica também varia ao longo da propagação (Liang, 2021; Sepehr; Beigy, 2022). A teoria dos grafos permite representar as relações entre usuários, identificar posições estruturalmente relevantes e descrever esse processo. Neste trabalho, a rede social será modelada como um grafo dirigido, e o grau de saída será adotado como medida operacional do alcance potencial dos iniciadores, sem pressupor equivalência com influência efetiva (Zareie; Sakellariou, 2023). As cascatas serão representadas separadamente, por meio dos usuários que compartilharam o conteúdo e das exposições que antecederam cada compartilhamento. Essa representação orientará a geração de cargas que considerem o volume, a distribuição e a evolução temporal dos eventos.

O problema desta pesquisa consiste em investigar como diferentes arquiteturas *backend* respondem ao estresse gerado por cascatas de informação em redes sociais. A investigação é orientada pela seguinte pergunta: como a topologia da rede e a concentração de influência em nós com alto grau de saída afetam o desempenho, a formação de filas, a utilização de recursos e a resiliência de duas composições arquiteturais, uma predominantemente síncrona e outra orientada a eventos, quando submetidas ao mesmo padrão simulado de viralização? A comparação considera uma arquitetura síncrona, baseada em PHP-FPM, Laravel e MySQL, com fluxo síncrono e persistência transacional linha a linha, e uma arquitetura orientada a eventos, baseada em PHP com Swoole/Hyperf, RabbitMQ e Neo4j, com processamento assíncrono, filas e ingestão em lotes por *workers*.

A relevância do estudo está em aproximar propriedades estruturais da rede das consequências observáveis na infraestrutura de software. Medidas como alcance, profundidade, largura e velocidade descrevem aspectos dependentes entre si e, por isso, requerem controle conjunto na comparação de cascatas (Juul; Ugander, 2021). Além disso, formas distintas de difusão podem produzir padrões diferentes de requisições e escritas persistentes (Sepehr; Beigy, 2022). A avaliação conjunta de grafos, carga e observabilidade pode, portanto, oferecer uma análise experimental mais precisa da degradação e da resiliência das arquiteturas, sem restringir a comparação ao tempo de resposta nem pressupor a superioridade de uma composição.

O objetivo geral é analisar comparativamente o comportamento de duas composições arquiteturais completas, uma síncrona e outra orientada a eventos, durante a simulação de cascatas de informação associadas à viralização em redes sociais. O estudo não busca isolar o efeito individual de cada tecnologia, mas comparar as composições sob condições equivalentes de carga. Para alcançá-lo, são definidos os seguintes objetivos específicos: i) modelar redes

e cascatas com diferentes distribuições de conectividade e concentrações de grau de saída; ii) implementar geradores de carga capazes de reproduzir picos controlados de compartilhamento; iii) submeter as duas arquiteturas ao mesmo estímulo; iv) coletar métricas de recursos, latência, *throughput*, erros, filas, persistência e rastreamento distribuído; e v) comparar saturação, degradação, drenagem das filas e recuperação após o pico. Os parâmetros e critérios que assegurarão a equivalência dos experimentos serão definidos na metodologia antes de sua execução.

O restante deste artigo está organizado da seguinte forma: o referencial teórico define os conceitos de redes, cascatas, arquiteturas e observabilidade empregados na pesquisa; a seção de trabalhos relacionados examina estudos próximos à comparação proposta; a metodologia descreve a classificação da pesquisa, o desenho experimental, o modelo de carga e os procedimentos de coleta e análise dos dados; a seção de desenvolvimento apresenta a implementação das arquiteturas e dos mecanismos de simulação e observabilidade; os resultados apresentam e discutem os dados obtidos nos experimentos; e, por fim, as considerações finais sintetizam as conclusões, as limitações e as possibilidades de trabalhos futuros.

2 REFERENCIAL TEÓRICO

Esta seção reúne os conceitos necessários para relacionar a estrutura das redes sociais ao comportamento das duas composições arquiteturais sob carga. A exposição parte da representação da rede e da propagação da informação, caracteriza os fluxos síncrono e orientado a eventos e, por fim, delimita os sinais e indicadores usados para observar a resposta dos sistemas.

2.1 Redes sociais e teoria dos grafos

Um grafo é uma estrutura formada por vértices e arestas: no domínio deste estudo, cada vértice representa um usuário e cada aresta dirigida representa uma relação pela qual um conteúdo pode alcançar outro usuário. O grau de um vértice corresponde ao número de arestas que incidem nele; em grafos dirigidos, o grau de saída contabiliza as conexões que partem do usuário. Redes que crescem por conexão preferencial podem concentrar muitas relações em poucos vértices, produzindo distribuições desiguais de conectividade (Piva; Ribeiro; Mata, 2021). A centralidade designa uma família de medidas empregadas para caracterizar a posição estrutural dos vértices. Neste trabalho, o grau de saída será uma variável operacional de centralidade, e o alcance indicará quantos usuários podem ser atingidos a partir de um iniciador. Influência, por sua vez, será entendida como a capacidade efetiva de induzir comportamentos, não como sinônimo automático de conectividade, pois atributos comportamentais e o modelo de difusão também afetam a propagação (Zareie; Sakellariou, 2023).

Propagação é o processo pelo qual a informação percorre as relações da rede. Uma cascata de informação é o subgrafo formado pelos compartilhamentos de um conteúdo e pelas

exposições que os antecedem. Sua profundidade é a maior distância, em gerações de compartilhamento, entre o iniciador e um participante; sua largura corresponde à quantidade de ocorrências em uma mesma geração. O alcance contabiliza os participantes ou expostos, enquanto a forma da cascata distingue, por exemplo, uma difusão ampla e rasa de outra distribuída por várias gerações. Essa distinção é necessária porque cascatas de tamanhos semelhantes podem possuir estruturas e mecanismos de difusão diferentes (Sepehr; Beigy, 2022; Juul; Ugander, 2021).

2.2 Viralização e efeito manada

Viralização corresponde, neste trabalho, à disseminação rápida e ampla de um conteúdo por sucessivos compartilhamentos. O chamado efeito manada descreve a tendência de uma pessoa adotar um comportamento após observar a adesão de outras, de modo que a exposição social pode alterar a probabilidade de compartilhar. A posição topológica do iniciador condiciona o público que pode ser alcançado, mas métricas puramente estruturais não explicam, isoladamente, a decisão de retransmitir. Estudos recentes mostram que fatores relacionados ao usuário, à mensagem e à geração da cascata modificam os efeitos de contágio, inclusive com variação ao longo da profundidade (Liang, 2021); por isso, alto grau de saída será tratado como alcance potencial, e não como prova de influência.

A literatura também adverte que tamanho, profundidade, largura e evolução temporal não devem ser interpretados como dimensões independentes. O pareamento apenas pelo tamanho pode ocultar diferenças relevantes na estrutura das cascatas (Juul; Ugander, 2021), enquanto medidas de virulência estrutural procuram representar como a difusão se distribui entre transmissões diretas e sucessivas gerações (Sepehr; Beigy, 2022). Assim, a carga experimental deverá controlar tanto o volume total quanto a distribuição temporal e topológica dos eventos. O uso do termo efeito manada, portanto, não pressupõe que toda exposição resulte em adesão; ele delimita o mecanismo social simulado que poderá produzir concentração de eventos em curto intervalo.

2.3 Arquitetura síncrona

Na comunicação síncrona por requisição e resposta, um cliente envia uma solicitação e aguarda a conclusão do processamento para receber o resultado. Revisões sobre padrões de comunicação em microsserviços distinguem esse acoplamento temporal dos padrões assíncronos e mostram que a escolha do padrão depende do contexto e de seus desafios operacionais (Karabey Aksakalli et al., 2021). Na primeira composição deste estudo, o PHP-FPM gerencia processos FastCGI em grupos de *workers*, com modos configuráveis de criação de processos e informações de estado apropriadas à operação sob carga (PHP DOCUMENTATION GROUP,

2026). O Laravel recebe a requisição, aplica as regras da operação e coordena o acesso ao MySQL antes de produzir a resposta.

A persistência relacional organiza os dados em tabelas e relacionamentos definidos por chaves. A expressão "linha a linha" identifica que cada evento de compartilhamento gera operações individuais de inserção ou atualização, ainda que várias operações possam pertencer à mesma transação. Uma transação delimita uma unidade de trabalho concluída por confirmação, que torna permanentes as alterações, ou por reversão, que as cancela (ORACLE, 2026). Nessa composição, o tempo percebido pelo cliente inclui o processamento da aplicação e a persistência necessária à resposta; sob aumento de concorrência, requisições podem aguardar processos, conexões ou recursos do banco. Essas relações constituem hipóteses a observar, não resultados antecipados.

2.4 Arquitetura orientada a eventos

Em uma arquitetura orientada a eventos, mudanças relevantes são representadas como eventos e publicadas para processamento por componentes desacoplados no tempo. Swoole fornece ao PHP um mecanismo de rede baseado em eventos assíncronos e corrotinas, sobre o qual o Hyperf estrutura a aplicação (SWOOLE, 2026). O produtor cria a mensagem; o RabbitMQ atua como intermediador e a armazena em uma fila; o consumidor a recebe; e um *worker*, isto é, um processo responsável por executar unidades de trabalho, realiza o processamento. A requisição pode terminar depois da aceitação do evento, sem aguardar toda a persistência subsequente, razão pela qual latência de aceitação e latência de processamento completo deverão ser medidas separadamente.

Processamento assíncrono significa que produção e consumo não precisam ocorrer simultaneamente. Quando a chegada supera temporariamente a capacidade dos consumidores, mensagens pendentes formam um *backlog*. O processamento em lote agrupa várias mensagens em uma operação de persistência, reduzindo a quantidade de interações com o banco, mas introduzindo critérios de tamanho e tempo para formar cada lote. No RabbitMQ, a confirmação do consumidor informa que uma entrega foi processada, enquanto a confirmação do publicador indica que o intermediador assumiu responsabilidade pela mensagem; são mecanismos distintos e necessários à definição explícita da confiabilidade (RABBITMQ, 2026).

O Neo4j representa entidades como nós e relações como conexões de primeira classe. Nesta composição, usuários e compartilhamentos podem ser persistidos segundo a topologia que será percorrida nas análises. Operações no grafo, nos índices ou no esquema ocorrem em transações, e atualizações muito grandes precisam ser divididas para limitar o consumo de memória (NEO4J, 2026). A ingestão em lotes não elimina, portanto, a fronteira transacional: ela altera a quantidade de eventos reunidos em cada unidade de persistência. A comparação será feita entre composições completas, sem atribuir antecipadamente eventual diferença a uma tecnologia isolada.

2.5 Observabilidade sob carga

Observabilidade é a capacidade de inferir o estado interno de um sistema a partir dos sinais que ele produz. Métricas são séries numéricas agregadas; *logs* são registros discretos de eventos com seu contexto; e rastreamento distribuído acompanha uma operação pelas etapas e componentes que a processam. A correlação temporal e por identificadores entre esses sinais permite relacionar alterações de desempenho a requisições, mensagens, *workers* e operações de persistência (Tzanettis et al., 2022). O OpenTelemetry será empregado como estrutura independente de fornecedor para gerar, coletar e exportar esses sinais, sem substituir os sistemas responsáveis por armazená-los e visualizá-los (OPEN TELEMETRY, 2026).

Sob carga, latência é o tempo transcorrido para concluir uma operação, e vazão é a quantidade de operações concluídas por unidade de tempo. Como médias podem ocultar ocorrências extremas, percentis como p50, p95 e p99 indicarão os limites abaixo dos quais se encontram, respectivamente, 50%, 95% e 99% das observações. A taxa de erros expressará a proporção de operações malsucedidas; saturação caracterizará o ponto em que um recurso se aproxima de sua capacidade e o aumento de demanda passa a intensificar espera ou falhas. Na composição assíncrona, o tamanho do *backlog* quantificará mensagens ainda não concluídas e o tempo de drenagem medirá o intervalo necessário para processar a fila remanescente após o encerramento do pico. Esses indicadores, juntamente com CPU, memória e duração da persistência, são variáveis planejadas para os experimentos e não constituem resultados já obtidos.

3 TRABALHOS RELACIONADOS

Abella, San Miguel e Ramasco (2023) analisaram o efeito do envelhecimento dos agentes na dinâmica temporal de cascatas por meio de um modelo de limiar para contágio complexo. O método incorporou diferentes mecanismos de envelhecimento e avaliou como eles alteram a evolução macroscópica do processo. O estudo contribui ao distinguir as condições de ocorrência da cascata de sua velocidade de propagação. Assim como este TCC, considera que a distribuição temporal dos eventos é relevante; diferentemente dele, concentra-se na dinâmica do modelo social, e não na resposta de arquiteturas *backend* ao padrão de carga resultante.

Cabane e Farias (2024) compararam empiricamente versões monolítica e orientada a eventos de uma mesma aplicação, submetendo-as a cargas e analisando CPU, memória, tempo de resposta, vazão e tráfego de rede. A contribuição está na avaliação conjunta de métricas e no tratamento estatístico das diferenças observadas. A semelhança com este TCC é a comparação experimental de composições arquiteturais sem pressupor superioridade de um estilo; a diferença está no estímulo, pois o presente estudo empregará cascatas derivadas de propriedades topológicas e também observará filas, drenagem e rastros distribuídos.

Kotiranta, Junkkari e Nummenmaa (2022) compararam Neo4j, MySQL e MariaDB por meio de consultas com estruturas e complexidades distintas, considerando também índices e

otimizações. O trabalho contribui ao demonstrar que o desempenho depende do tipo de consulta e da configuração, sem sustentar a superioridade universal de um modelo de dados. A semelhança está na presença de persistência relacional e orientada a grafos; a diferença é que o presente TCC avaliará ingestão durante cascatas de carga em arquiteturas completas, e não o desempenho isolado de consultas entre bancos.

Em síntese, os três estudos oferecem fundamentos para controlar a dinâmica temporal, comparar estilos arquiteturais e interpretar diferenças entre bancos. No conjunto selecionado, entretanto, não foi identificada uma avaliação que transforme propriedades topológicas de cascatas sociais em estímulo controlado para comparar uma composição síncrona e outra orientada a eventos. Este trabalho busca atender a essa lacuna ao relacionar a propagação simulada aos sinais de infraestrutura, sem antecipar resultados nem atribuir eventual comportamento a uma tecnologia isolada.

4 METODOLOGIA

Esta seção descreve a classificação da pesquisa, as etapas planejadas, as ferramentas empregadas e o desenho experimental adotado para a comparação das arquiteturas. A metodologia foi estruturada para permitir a replicação dos experimentos e a análise controlada dos dados coletados.

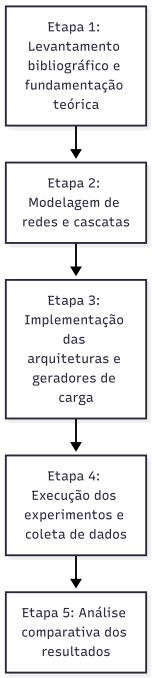
4.1 Classificação da pesquisa

A pesquisa pode ser classificada sob diferentes critérios, conforme as dimensões propostas por Gil (2022) e Wazlawick (2021). Quanto à abordagem, o estudo é quantitativo, pois mensura variáveis numéricas como latência, vazão, utilização de recursos, tamanho de filas e tempo de drenagem, submetendo-as a tratamento estatístico. Quanto à natureza, trata-se de uma pesquisa aplicada, uma vez que visa gerar conhecimento diretamente utilizável na avaliação e no projeto de sistemas backend submetidos a estresse. Quanto aos objetivos, a pesquisa é explicativa, pois busca identificar fatores que determinam o comportamento observado nas arquiteturas e as relações entre variáveis de carga, topologia e desempenho. Quanto aos procedimentos técnicos, o trabalho pode ser classificado como experimental, pois submeterá duas composições arquiteturais a condições controladas de estímulo para observar e comparar seus comportamentos.

4.2 Etapas da pesquisa

O desenvolvimento do trabalho está estruturado em cinco etapas, ilustradas na Figura 1. A primeira etapa consiste no levantamento bibliográfico e na fundamentação teórica, abrangendo teoria dos grafos, cascatas de informação, arquiteturas backend e observabilidade. A segunda etapa compreende a modelagem de redes sociais como grafos dirigidos e a especificação de cascatas com diferentes distribuições de conectividade e concentrações de grau de saída. A terceira etapa envolve a implementação das duas arquiteturas, a configuração do ambiente de execução e o desenvolvimento de geradores de carga capazes de reproduzir picos controlados de compartilhamento. A quarta etapa corresponde à execução dos experimentos, com coleta sistemática de métricas de recursos, latência, vazão, erros, filas, persistência e rastreamento distribuído. Por fim, a quinta etapa dedica-se à análise comparativa dos resultados, à identificação de padrões de degradação e recuperação e à síntese das conclusões.

Figura 1 – Fluxograma das etapas da pesquisa



Fonte: Elaborada pelo autor

4.3 Ferramentas e tecnologias

O ambiente de desenvolvimento e experimentação é constituído pelas ferramentas e tecnologias apresentadas no Quadro 1. A escolha de cada componente justifica-se pela aderência aos objetivos do trabalho e pela disponibilidade de suporte a instrumentação e observabilidade.

Quadro 1 – Ferramentas e tecnologias do ambiente experimental

Categoria	Ferramenta	Função
Geração de carga	k6	Simulação de requisições HTTP
Backend assíncrono	PHP Swoole / Hyperf	Servidor orientado a eventos
Backend síncrono	PHP-FPM / Laravel	Servidor síncrono
Banco de dados (grafos)	Neo4j	Persistência orientada a grafos
Banco de dados (relacional)	MySQL	Persistência relacional
Message broker	RabbitMQ	Intermediação de mensagens
Coleta de métricas	Prometheus	Armazenamento de séries temporais
Visualização	Grafana	Dashboards e alertas
Rastreamento distribuído	OpenTelemetry	Geração e exportação de rastros
Conteinerização	Docker	Isolamento e orquestração de serviços

textbfFonte: Elaborado pelo autor

O k6 será utilizado para gerar cargas que simulam cascatas de compartilhamento, permitindo controlar o volume, a distribuição temporal e a concentração de requisições em curtos intervalos. O PHP-FPM e o Laravel comporão a arquitetura síncrona, enquanto o Swoole e o Hyperf comporão a arquitetura orientada a eventos. O MySQL receberá a persistência da arquitetura síncrona, e o Neo4j, da arquitetura assíncrona. O RabbitMQ atuará como intermediador de mensagens na arquitetura orientada a eventos. Prometheus, Grafana e OpenTelemetry constituirão a plataforma de observabilidade, responsável pela coleta, armazenamento, visualização e correlação dos sinais produzidos pelos sistemas sob teste. O Docker será empregado para isolar e orquestrar os componentes, garantindo reprodutibilidade do ambiente experimental.

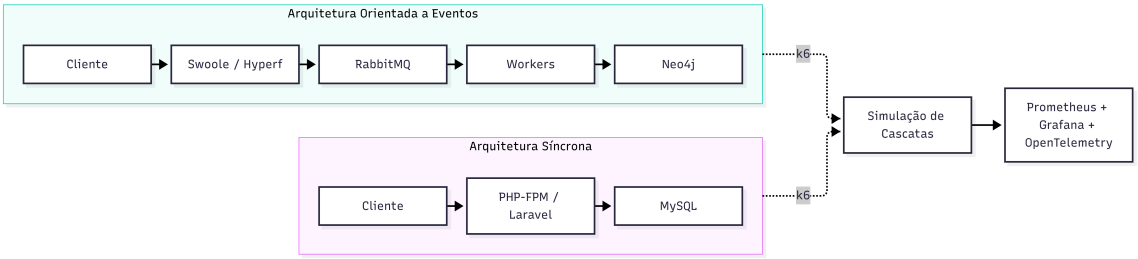
4.4 Desenho experimental

O desenho experimental está centrado na simulação de cascatas de informação derivadas de propriedades topológicas de redes sociais. Redes serão modeladas como grafos dirigidos, e cascatas serão geradas a partir de nós selecionados com diferentes graus de saída, produzindo padrões de propagação que variam em profundidade, largura e velocidade. O gerador de carga, implementado com k6, produzirá requisições que representam compartilhamentos, respeitando a evolução temporal das cascatas e a concentração de eventos em nós com alto grau.

As duas arquiteturas serão submetidas ao mesmo estímulo, definido por parâmetros como tamanho da rede, distribuição de grau, profundidade máxima da cascata, probabilidade de compartilhamento e ritmo de propagação. A cada execução, serão coletadas métricas de latência (p50, p95, p99), vazão, taxa de erros, utilização de CPU e memória, tamanho do backlog de mensagens e tempo de drenagem das filas. Rastros distribuídos permitirão correlacionar requisições, mensagens e operações de persistência ao longo do tempo.

A Figura 2 apresenta, de forma esquemática, as duas composições arquiteturais que serão comparadas. A arquitetura síncrona processa cada requisição de forma bloqueante, com persistência imediata. A arquitetura orientada a eventos desacopla a aceitação da requisição do processamento subsequente, utilizando filas e workers para ingestão em lotes.

Figura 2 – Diagrama comparativo das arquiteturas



Fonte: Elaborada pelo autor

Os experimentos serão organizados em cenários que variam a concentração de grau de saída dos iniciadores, o tamanho das cascatas e a intensidade do pico. Cada cenário será repetido múltiplas vezes para permitir análise estatística. Os parâmetros específicos, como quantidade de usuários simulados, duração dos testes e configurações de hardware, serão definidos na seção de desenvolvimento e mantidos constantes entre as duas arquiteturas para assegurar a equivalência da comparação.

REFERÊNCIAS

ABELLA, D.; San Miguel, M.; RAMASCO, J. J. Aging in binary-state models: The threshold model for complex contagion. **Physical Review E**, v. 107, n. 2, p. 024101, 2023. Disponível em: https://doi.org/10.1103/PhysRevE.107.024101. Acesso em: 30 ago. 2026.

CABANE, H.; FARIAS, K. On the impact of event-driven architecture on performance: An exploratory study. **Future Generation Computer Systems**, v. 153, p. 52–69, 2024. Disponível em: https://doi.org/10.1016/j.future.2023.10.021. Acesso em: 30 ago. 2026.

GIL, A. C. **Como elaborar projetos de pesquisa**. 7. ed. Barueri: Atlas, 2022.

JUUL, J. L.; UGANDER, J. Comparing information diffusion mechanisms by matching on cascade size. **Proceedings of the National Academy of Sciences**, v. 118, n. 46, p. e2100786118, 2021. Disponível em: https://doi.org/10.1073/pnas.2100786118. Acesso em: 30 ago. 2026.

Karabey Aksakalli, I. et al. Deployment and communication patterns in microservice architectures: A systematic literature review. **Journal of Systems and Software**, v. 180, p. 111014, 2021. Disponível em: https://doi.org/10.1016/j.jss.2021.111014. Acesso em: 21 set. 2026.

KOTIRANTA, P.; JUNKKARI, M.; NUMMENMAA, J. Performance of graph and relational databases in complex queries. **Applied Sciences**, v. 12, n. 13, p. 6490, 2022. Disponível em: https://doi.org/10.3390/app12136490. Acesso em: 30 ago. 2026.

LIANG, H. Decreasing social contagion effects in diffusion cascades: Modeling message spreading on social media. **Telematics and Informatics**, v. 62, p. 101623, 2021. Disponível em: https://doi.org/10.1016/j.tele.2021.101623. Acesso em: 30 ago. 2026.

NEO4J. **Database Transactions**. 2026. Disponível em: https://neo4j.com/docs/operations-manual/current/database-internals/transaction-management/. Acesso em: 21 set. 2026.

OPEN TELEMETRY. **What is OpenTelemetry?** 2026. Disponível em: https://opentelemetry-.io/docs/what-is-opentelemetry/. Acesso em: 30 ago. 2026.

ORACLE. **MySQL 8.4 Reference Manual: START TRANSACTION, COMMIT, and ROLLBACK Statements**. 2026. Disponível em: https://dev.mysql.com/doc/refman/8.4/en-/commit.html. Acesso em: 21 set. 2026.

PHP DOCUMENTATION GROUP. **FastCGI Process Manager (FPM)**. 2026. Disponível em: https://www.php.net/manual/en/install.fpm.php. Acesso em: 21 set. 2026.

PIVA, G. G.; RIBEIRO, F. L.; MATA, A. S. Networks with growth and preferential attachment: Modelling and applications. **Journal of Complex Networks**, v. 9, n. 1, p. cnab008, 2021. Disponível em: https://doi.org/10.1093/comnet/cnab008. Acesso em: 30 ago. 2026.

RABBITMQ. **Consumer Acknowledgements and Publisher Confirms**. 2026. Disponível em: https://www.rabbitmq.com/docs/confirms. Acesso em: 21 set. 2026.

SEPEHR, A.; BEIGY, H. Structural virality estimation and maximization in diffusion networks. **Expert Systems with Applications**, v. 206, p. 117657, 2022. Disponível em: https://doi.org-/10.1016/j.eswa.2022.117657. Acesso em: 30 ago. 2026.

SWOOLE. **Swoole Documentation**. 2026. Disponível em: https://wiki.swoole.com/en/. Acesso em: 21 set. 2026.

TZANETTIS, I. et al. Data fusion of observability signals for assisting orchestration of distributed applications. **Sensors**, v. 22, n. 5, p. 2061, 2022. Disponível em: https://doi.org/10.3390-/s22052061. Acesso em: 30 ago. 2026.

WAZLAWICK, R. S. **Metodologia de pesquisa para ciência da computação**. 3. ed. Rio de Janeiro: LTC, 2021.

ZAREIE, A.; SAKELLARIOU, R. Influence maximization in social networks: A survey of behaviour-aware methods. **Social Network Analysis and Mining**, v. 13, n. 1, p. 78, 2023. Disponível em: https://doi.org/10.1007/s13278-023-01078-9. Acesso em: 30 ago. 2026.